

Component	Constraints	Possible Solutions	Ideal Solution	Technical Components	Justification
1. Cloud Infrastructure (IaaS)	- Open-source mandatory - Multi-tenant architecture - Private deployment - Commodity hardware - On-demand virtualized resources (servers, storage, containers) - Automatic auto-scaling based on workload	OpenStack CloudStack OpenNebula Proxmox VE	OpenStack	- Nova (Compute) - Cinder (Block Storage) - Swift (Object Storage) - Neutron (Networking) - Keystone (Identity) - Horizon (Dashboard) - Octavia (Load Balancer)	- De facto standard for open-source private clouds - Mature ecosystem with massive community support - Modularity enabling progressive deployment - Extensive hardware compatibility - AWS-compatible APIs
2. Virtualization	- Linux compatibility - Live migration support - Near-native performance - Integration with OpenStack - Modern hardware support (Intel VT-x, AMD-V) - Production stability and reliability - Standardized API for interoperability	KVM Xen VMware ESXi LXD	KVM (Kernel-based Virtual Machine)	- KVM kernel module - Libvirt API - QEMU	- Integrated directly into Linux kernel - Stable and fast live migration - Standard Libvirt API - Native compatibility with OpenStack Nova - Support nested virtualization - Open-source with broad industrial adoption
3. Infrastructure Automation	- Text templates to describe infrastructure - Specify relationships between resources - Advanced features (HA, auto-scaling, nested stacks) - Human-readable - Version-controllable (Git) - Standard format - Open-source - Agentless	Ansible + OpenStack Heat Terraform Puppet Chef SaltStack	Ansible	- Ansible (Playbooks YAML, SSH) - Heat (Templates HOT, Auto-scaling) - Ansible Galaxy - AutoScalingGroup with Ceilometer alarms	- Ansible: Simple YAML syntax and agentless via SSH - Guaranteed idempotence - Ansible Galaxy for community roles - Native OpenStack modules integration - Heat: YAML/JSON templates version-controllable with Git - Standard OpenStack HOT format - Dependency management with DependsOn - Rolling updates support for zero-downtime
4. Containerization	- Isolation per client/organization (multi-tenant) - Portability across different OS and hardware - Reduced overhead compared to VMs - Near-native performance - Multi-OS support (Linux, Windows) - Mature and standardized ecosystem	Docker + Kubernetes Namespaces LXC/LXD Podman rkt (Rocket) Containerd alone	Docker	- Docker Engine - Kubernetes Namespaces - Resource quotas - Network policies - Docker Registry Hub	- Docker: De facto industry standard - Strong isolation via namespaces (PID, Network, Mount, IPC, UTS) - Multi-OS support - Image layering for storage efficiency - Registry Hub Docker with millions of images - K8s Namespaces: Strong logical isolation between tenants - Resource quotas prevent resource exhaustion - NetworkPolicies for network isolation - RBAC per-namespace for granular access
5. Container Orchestration	- Centralized orchestration of all instances - Service Discovery and Load Balancing - Automatic self-healing - Deployment patterns and templates - Containers as a Service delivery - Guaranteed high availability - Automatic scalability	Kubernetes Docker Swarm Apache Mesos Nomad (HashiCorp)	Kubernetes	- Control Plane (API Server, Scheduler, Controller Manager, etcd) - Workers (Kubelet, Kube-proxy) - Services - Deployments - StatefulSets - Helm	- Industry orchestration standard (CNCF graduated project) - Resilient distributed architecture (etcd clustering) - Automatic service discovery via CoreDNS - Integrated L4 load balancing (kube-proxy) - Self-healing: ReplicaSets + liveness/readiness
6. Configuration Management	- Automated provisioning, configuration and deployment - Agentless - Monitoring and alerting automation - Declarative configuration	Ansible Puppet Chef SaltStack Terraform	Ansible	- Ansible Core - Playbooks YAML - Roles & Collections - Dynamic Inventory - Ansible Galaxy	- Agentless: SSH/WinRM communication without daemon - Enhanced security (SSH standard port 22) - Simple syntax: Readable YAML, low learning curve